



UNIVERSITAS  
GADJAH MADA

---

# Configuration Drift pada Deployment Aplikasi Multi-Service di Kluster Kubernetes

---

Evaluasi Empiris Paradigma Manual dan GitOps  
(Studi Kasus InvenioRDM)

Muhammad Argya Vityasy

NIM: 23/522547/PA/22475

Dosen Pembimbing: Dr. techn. Guntur Budi Herwanto, S.Kom., M.Cs.

Dosen Penguji 1: Dr. techn. Kabul Kurniawan, S.Kom., M.Cs.

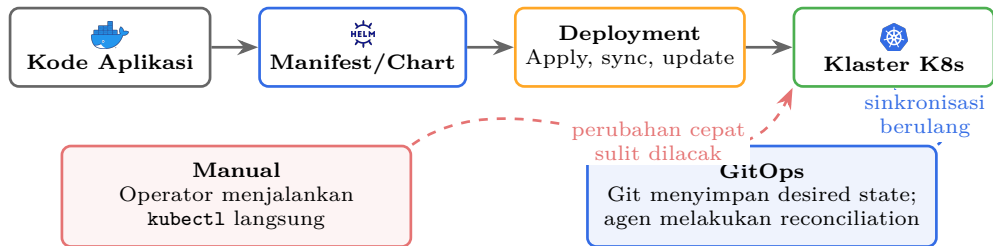
Dosen Penguji 2: Aina Musdholifah, S.Kom., M.Kom., Ph.D.

ugm.ac.id

LOCALLY ROOTED, GLOBALLY RESPECTED

# Konteks: Dari Deployment ke GitOps

Deployment membawa kode dan konfigurasi menjadi layanan yang berjalan di kluster.



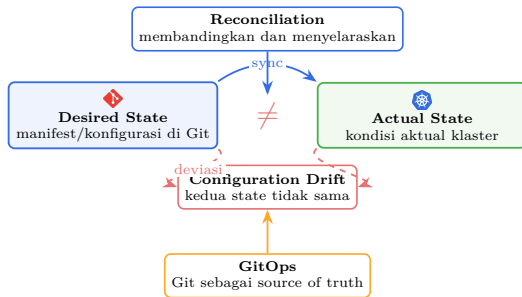
GitOps muncul untuk menjawab kebutuhan deployment yang deklaratif, terlacak, dan terus direkonsiliasi.

# Istilah Kunci dalam Penelitian

## Terminologi utama

- **Deployment:** proses menjalankan/memperbarui aplikasi di infrastruktur.
- **Kubernetes:** platform orkestrasi kontainer.
- **Desired state:** kondisi yang dideklarasikan di manifest/Git.
- **Actual state:** kondisi aktual di klaster.
- **Reconciliation:** penyelarasan state.
- **Configuration drift:** selisih antara desired dan actual state.

Istilah “drift” selalu merujuk pada selisih antara state yang dideklarasikan dan state aktual klaster.



# Mengapa Masalah Ini Penting?

## Situasi yang mudah terjadi

- Aplikasi Kubernetes punya banyak komponen.
- Perubahan cepat dilakukan langsung di kluster.
- Konfigurasi tertulis tidak ikut diperbarui.

## Dampaknya

- Tim tidak yakin state mana yang benar.
- Pemulihan dan analisis akar masalah melambat.
- Jejak audit menjadi tidak lengkap.

Masalah utamanya bukan hanya aplikasi bermasalah, tetapi hilangnya kepastian tentang state sistem.



# Configuration Drift: Definisi dan Bukti

## Definisi (Pohjola, 2025)

State aktual kluster tidak lagi sesuai dengan state yang dideklarasikan.

### Penyebab umum

1. Perubahan langsung tanpa *source of truth*
2. Prosedur deployment tidak konsisten
3. Tidak ada deteksi otomatis

## Bukti empiris

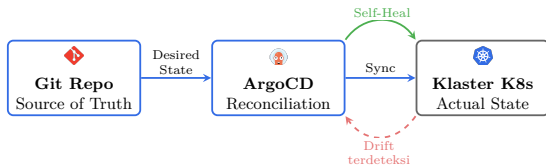
- 71% dari 2.260 skrip K8s mengandung *misconfiguration* bermakna (Zhang et al., 2026).
- 11 kategori *security misconfiguration* berulang pada manifest K8s (Rahman et al., 2023).
- Drift berdampak pada keandalan, keamanan, dan proses pemulihan layanan.

**Karena itu, drift perlu diukur sebagai fenomena operasional, bukan hanya dijelaskan sebagai konsep.**

# GitOps sebagai Respons terhadap Drift

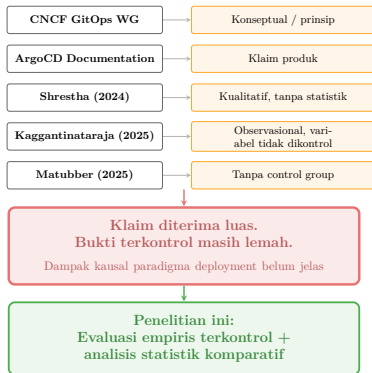
- Git menyimpan *desired state*.
- Agen di kluster membandingkan Git dengan state aktual.
- Saat ada drift, sistem dapat mendeteksi dan mengoreksinya.

## Prinsip GitOps



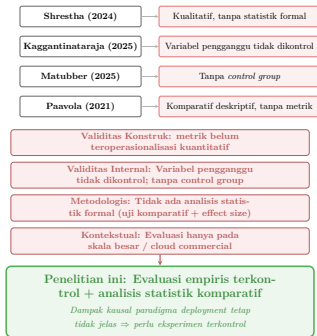
Pertanyaan ilmiahnya bukan “apakah GitOps populer?”, melainkan seberapa berbeda respons operasionalnya saat drift terjadi.

# Klaim Industri vs. Bukti Ilmiah



**Klaimnya kuat di praktik, tetapi bukti terkontrol tentang dampak kausalnya masih terbatas.**

# Apa yang Membedakan Penelitian Ini?



**Penelitian ini mengisi empat celah: metrik yang jelas, variabel yang dikontrol, uji statistik formal, dan konteks klaster akademis.**



Sejauh mana deployment manual dengan `kubectl` dan GitOps dengan ArgoCD menghasilkan respons operasional yang berbeda ketika menghadapi *configuration drift* terkontrol?

1. **R1 -- Konsistensi:** apakah deployment kembali ke state yang diharapkan?
2. **R2 -- Waktu:** berapa lama drift dideteksi, dipulihkan, dan diidentifikasi?
3. **R3 -- Traceability:** seberapa lengkap jejak perubahan dapat direkonstruksi?

1. Satu klaster Kubernetes Rancher: 3 node, 24 inti CPU,  $\sim 23$  Gi memori.
2. InvenioRDM sebagai subjek uji, bukan objek penelitian.
3. Tujuh skenario *drift* terkontrol (A--G).
4. Dua paradigma: deployment manual (`kubectl`) dan GitOps (ArgoCD).
5. Analisis: *Mann--Whitney U*,  $\alpha = 0,05$ , dan ukuran efek.
6. Di luar lingkup: *multi-cluster* dan Git tanpa agen *reconciliation*.

# Tujuan dan Manfaat Penelitian

## Tujuan

1. Membandingkan konsistensi deployment.
2. Membandingkan waktu deteksi, pemulihan, dan identifikasi akar masalah.
3. Mendokumentasikan *traceability* operasional.

## Teoritis

Bukti empiris tentang dampak drift pada dua paradigma deployment.

## Metodologis

Protokol yang dapat direplikasi: skenario, metrik, dan analisis.

## Praktis

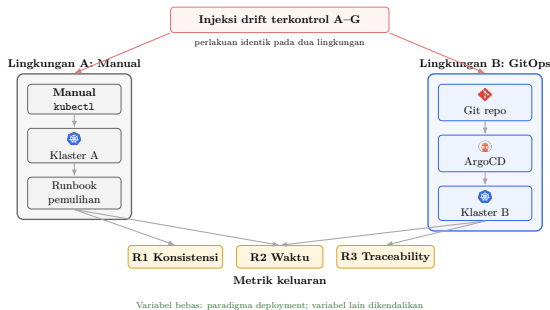
Masukan operasional bagi pengelola klaster Kubernetes.

**Kontribusi utama: mengukur dampak operasional pilihan paradigma deployment, bukan membuktikan satu teknologi selalu unggul.**

# Penelitian Terkait dan Pembeda

| Sumber                                                      | Temuan Utama                                                           | Kelemahan Metodologis                                                                |
|-------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Shrestha (2024)<br>Kaggantinaraja (2025)<br>Matubber (2025) | GitOps membantu deteksi drift<br>Deklaratif lebih cepat dan aman       | Kualitatif; tanpa statistik formal<br>Variabel pengganggu tidak dikontrol            |
| Paavola (2021)                                              | Reconciliation time dapat diukur<br>ArgoCD cocok untuk banyak aplikasi | Tanpa <i>control group</i> manual<br>Komparatif deskriptif; tanpa metrik kuantitatif |

**Pembeda: eksperimen terkontrol, variabel pengganggu dikendalikan, analisis statistik formal, dan metrik didefinisikan sebelum eksperimen.**



## Variabel kontrol

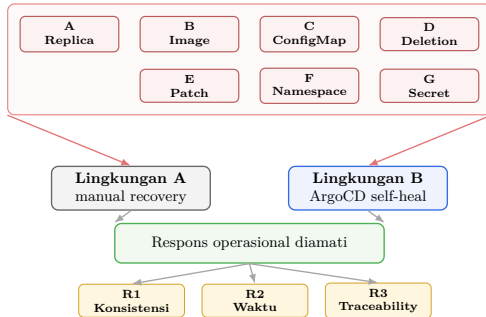
Workload, drift, klaster, dan prosedur pengukuran dibuat sama. Variabel yang dibedakan hanya paradigma deployment.

## Justifikasi workload

InvenioRDM dipakai karena dependensinya kompleks, komponen infrastrukturnya luas, dan *open-source*. Ia adalah workload uji, bukan objek penelitian.

# Skenario Configuration Drift Terkontrol

Tujuh skenario drift yang sama diberikan ke dua paradigma



Setiap skenario diinjeksi secara identik pada lingkungan manual dan GitOps, lalu respons operasionalnya diukur.

# Metrik Pengukuran dan Operasionalisasi

| Metrik                | Operasionalisasi                                    | Output                                                  |
|-----------------------|-----------------------------------------------------|---------------------------------------------------------|
| R1<br>Konsistensi     | Persentase resource yang cocok dengan desired state | Nilai % pada baseline, saat drift, dan setelah recovery |
| R2a<br>Detection time | $t_d - t_0$                                         | Waktu dari injeksi hingga drift terdeteksi              |
| R2b<br>Recovery time  | $t_2 - t_0$                                         | Waktu dari injeksi hingga state kembali baseline        |
| R2c<br>Root cause ID  | $t_{rci} - t_d$                                     | Waktu dari deteksi hingga penyebab teridentifikasi      |
| R3<br>Traceability    | Bukti Git history, audit log, dan diff              | Deskripsi kelengkapan rekonstruksi perubahan            |

R1 dan R2 dibandingkan antar paradigma; R3 dideskripsikan karena sifat audit trail berbeda secara struktural.

Root Cause Identification Time selesai ketika kategori drift dan sumber daya terdampak disebutkan dengan benar.

# Ancaman terhadap Validitas dan Mitigasi

| Ancaman               | Mitigasi dalam rancangan                                    |
|-----------------------|-------------------------------------------------------------|
| Operator tunggal      | Recovery runbook dan timestamp otomatis                     |
| Efek pembelajaran     | Urutan skenario diacak dan ada practice run                 |
| Instrumentasi berubah | Versi Kubernetes, ArgoCD, dan workload dikunci              |
| Satu workload         | InvenioRDM dipilih sebagai workload multi-service realistis |
| Satu alat GitOps      | ArgoCD digunakan sebagai implementasi GitOps yang umum      |

**Klaim penelitian dibatasi pada evaluasi empiris terkontrol dalam konteks yang didefinisikan.**



## Metode analisis

- Statistik deskriptif per metrik.
- *Mann--Whitney U*,  $\alpha = 0,05$ .
- Ukuran efek: Cohen's  $d$  atau Cliff's  $\delta$ .
- R3 dianalisis secara deskriptif.

## Justifikasi

- *Mann--Whitney*: data waktu belum tentu normal dan sampel per skenario relatif kecil.
- $n = 10$  per skenario: total  $7 \times 10 = 70$  observasi per lingkungan untuk R2.

# Hasil yang Mungkin dan Maknanya

## **GitOps lebih baik**

Klaim industri mendapat dukungan empiris pada konteks terkontrol.

## **Tidak berbeda bermakna**

Klaim keunggulan otomatis GitOps perlu dibatasi pada konteks ini.

## **Manual unggul pada metrik tertentu**

Hasil menunjukkan batas praktis otomatisasi GitOps pada metrik tertentu.

**Apa pun arahnya, kontribusi penelitian adalah bukti yang dapat diperiksa, bukan opini adopsi teknologi.**

*“Penelitian ini mengukur bagaimana  
pilihan paradigma deployment  
mengubah respons operasional terhadap  
configuration drift, bukan membuktikan  
bahwa satu teknologi selalu menang.”*



UNIVERSITAS  
GADJAH MADA

LOCALLY ROOTED, GLOBALLY RESPECTED

[ugm.ac.id](http://ugm.ac.id)

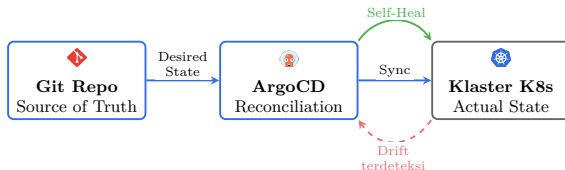
- **Deployment:** mengelola siklus hidup aplikasi pada infrastruktur, termasuk konfigurasi, *scaling*, dan pembaruan.
- **Kubernetes:** platform *container orchestration* untuk menjalankan aplikasi *distributed* (Burns et al., 2016).
- **GitOps:** paradigma deployment deklaratif dengan Git sebagai *single source of truth* dan agen *software* yang melakukan *reconciliation* secara kontinu (CNCF GitOps WG, 2021).
- **Configuration Drift:** situasi di mana *state* aktual menyimpang dari *state* yang dideklarasikan (Pohjola, 2025).

# Backup: ArgoCD dan Mekanisme Reconciliation

- Git sebagai *single source of truth*
- Konfigurasi dideklarasikan secara deklaratif
- Agen *software* melakukan *reconciliation*
- Drift menjadi *observable* dan dapat dipulihkan otomatis

ArgoCD terdiri dari tiga komponen: *application controller*, *repo server*, dan *Redis cache*.

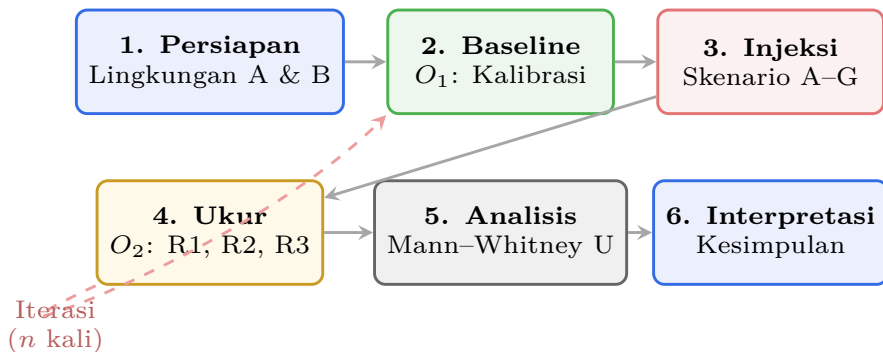
## Prinsip GitOps



# Backup: Detail Skenario Drift

| ID | Skenario            | Contoh perintah                                    |
|----|---------------------|----------------------------------------------------|
| A  | Perubahan replika   | <code>kubectl scale deployment --replicas=5</code> |
| B  | Perubahan image     | <code>kubectl edit deployment</code>               |
| C  | Perubahan ConfigMap | <code>kubectl edit configmap</code>                |
| D  | Resource deletion   | <code>kubectl delete deployment</code>             |
| E  | Patch manual        | <code>kubectl patch</code>                         |
| F  | Namespace salah     | <code>kubectl apply -f ... -n wrong-ns</code>      |
| G  | Perubahan Secret    | <code>kubectl patch secret</code>                  |

# Backup: Bagan Alir Eksperimen



Tahapan mengikuti pre-test – treatment – post-test: baseline, injeksi drift, pengukuran, analisis, interpretasi.

# Backup: Analisis Statistik dan Ukuran Sampel

## Analisis

- Statistik deskriptif per metrik.
- Mann--Whitney U,  $\alpha = 0,05$ .
- Ukuran efek: Cohen's  $d$  atau Cliff's  $\delta$ .
- R3 dianalisis secara deskriptif.

## Mengapa Mann--Whitney?

- Data waktu sistem belum tentu normal.
- Sampel per skenario relatif kecil.
- Uji non-parametrik tidak mensyaratkan normalitas.

## Justifikasi ukuran sampel

- Power = 0.80,  $\alpha = 0,05$ , target efek besar ( $d = 1,0$ ).
- Minimum  $n = 17$  per grup secara total.
- R2:  $7 \times 10 = 70$  observasi per lingkungan.
- R1:  $7 \times 5 = 35$  observasi per lingkungan.
- R3: deskriptif, tidak memerlukan ukuran sampel statistik.

## Mengapa $n = 10$ per skenario?

- Studi sistem berulang dan layak dalam batasan eksperimen.



# Backup: Ancaman terhadap Validitas (Detail)

## Validitas internal

- Seleksi lingkungan
- Operator tunggal
- Efek pembelajaran
- Perubahan instrumentasi

## Validitas konstruk

- Definisi drift dibatasi tujuh skenario
- Traceability bersifat deskriptif

## Mitigasi

- Klaster dan workload disamakan
- Runbook dan timestamp otomatis
- Urutan skenario diacak
- Versi software dikunci

## Validitas eksternal

- Hasil dibatasi pada konteks workload dan klaster yang dijelaskan.

# Backup: Definisi Operasional Root Cause Identification Time

**Pertanyaan: Bagaimana Anda menentukan secara objektif kapan akar masalah telah diidentifikasi?**

**Definisi operasional:**

- Selesai ketika operator menyebutkan **(1)** kategori drift dan **(2)** sumber daya terdampak, lalu jawaban diverifikasi terhadap skenario injeksi yang diketahui.

**Mengapa ini objektif:**

- Skenario sudah didefinisikan sebelum eksperimen.
- Lingkungan B: ArgoCD menampilkan *diff*.
- Lingkungan A: operator memakai `kubectl describe` dan `log`.
- *Timestamp* dicatat saat kategori dan sumber daya benar disebutkan.
- Prinsipnya mirip verifikasi *known-answer*.

# Backup: Mengapa Traceability (R3) Dideskripsikan, Bukan Diuji Statistik?

**Pertanyaan:** Jika Anda sudah tahu jawabannya, mengapa mengukur R3?

**Jawaban:**

- Tujuan R3 bukan *hypothesis testing*.
- R3 mendokumentasikan karakteristik operasional yang menjelaskan hasil R1 dan R2.
- GitOps menyediakan *commit history* dan *diff*; manual deployment bergantung pada log yang lebih parsial.
- Karena perbedaannya bersifat struktural, R3 dipakai untuk interpretasi, bukan uji hipotesis terpisah.

# Backup: Mengapa InvenioRDM, Bukan Workload Lain?

Pertanyaan: Mengapa tidak Online Boutique, Sock Shop, atau WordPress?

Tiga kriteria pemilihan:

1. **Dependensi kompleks:** 16+ komponen yang saling bergantung dan memiliki urutan deployment.
2. **Infrastruktur luas:** database, *search*, *cache*, *object storage*, *ingress*, TLS, *secrets*, monitoring, dan backup.
3. **Reprodusibilitas akademis:** *open-source*, dikembangkan CERN, dan digunakan di institusi akademis.

InvenioRDM adalah workload uji; klaim penelitian tetap dibatasi pada workload multi-service dengan karakteristik serupa.

# Backup: Ancaman Operator Tunggal dan Mitigasinya

Pertanyaan: Operator Anda juga peneliti. Mengapa ini bukan ancaman serius?

## Empat mitigasi:

1. Operator mengikuti *recovery runbook* yang telah ditentukan sebelumnya --- tidak ada keputusan subjektif selama *recovery*.
2. Seluruh *timestamp* direkam oleh skrip otomasi, bukan oleh penilaian manusia.
3. Metrik yang tidak bergantung pada operator (R1 konsistensi, R2 deteksi otomatis ArgoCD) memberikan pengukuran objektif.
4. Operator tunggal justru **mengurangi** varians antar-*run*, sehingga memperkuat validitas internal perbandingan antar paradigma.

Batasan ini diakui secara eksplisit sebagai ancaman validitas eksternal dan direkomendasikan untuk replikasi dengan operator berbeda.

# Backup: Apa Jika Tidak Ada Perbedaan Bermakna?

Pertanyaan: Jika hasil menunjukkan tidak ada perbedaan signifikan secara statistik, apakah itu berarti GitOps tidak memberikan nilai operasional?

## Tiga interpretasi:

1. **Hasil tidak signifikan dengan efek kecil:** Kedua paradigma menghasilkan hasil yang sebanding dalam konteks terkontrol ini. Ini sendiri adalah temuan yang layak dilaporkan.
2. **Hasil tidak signifikan dengan efek sedang/besar:** Efek ada tetapi varians tinggi; diperlukan lebih banyak pengulangan. Ukuran efek tetap memberikan informasi praktis.
3. **R3 tetap memberikan nilai struktural:** Meskipun R1 dan R2 tidak menunjukkan perbedaan signifikan, *traceability* (R3) memberikan nilai operasional yang terjamin secara struktural oleh GitOps.

# Backup: Bukankah GitOps Jelas Lebih Baik?

**Pertanyaan: Bukankah sudah jelas bahwa GitOps mempercepat pemulihan drift?**

**Jawaban:**

- Klaim bahwa GitOps membantu drift memang kuat, tetapi tetap perlu bukti terkontrol.
- Studi sebelumnya cenderung observasional, kualitatif, atau tanpa *control group*.
- Penelitian ini menanyakan **seberapa besar** perbedaannya dan dalam kondisi apa.
- Tanpa kontrol, efek bisa berasal dari operator, workload, atau konfigurasi kluster.
- Eksperimen ini mengisolasi variabel paradigma deployment.

# Backup: Apakah Anda Hanya Mengukur Perhatian Manusia?

Pertanyaan: Apakah perbedaan yang Anda ukur berasal dari GitOps atau dari keahlian operator?

Jawaban:

- Kemampuan deteksi adalah bagian dari paradigma deployment dan sengaja dimasukkan.
- Pada *manual deployment*, operator **adalah** mekanisme deteksi.
- Pada GitOps, ArgoCD **adalah** mekanisme deteksi.
- Keduanya adalah properti inheren dari paradigma masing-masing.
- Eksperimen membandingkan paradigma lengkap, bukan hanya kecepatan operator.
- Operator mengikuti runbook dengan *timestamp* otomatis.



# Backup: Mengapa Tujuh Skenario dan Pembobotan Sama?

**Pertanyaan: Mengapa tujuh skenario cukup? Mengapa dibobot sama?**

**Mengapa tujuh skenario:**

- Skenario diturunkan dari taksonomi Zhang et al. (2026) dan pola operasional Pohjola (2025).
- Kategori mencakup *scaling*, *image*, konfigurasi, penghapusan, *patching*, *namespace*, dan *secret*.
- Tidak mengklaim menutupi seluruh ruang drift, tetapi mewakili kategori operasional umum.

**Mengapa pembobotan sama:**

- Pembobotan sama adalah asumsi awal yang konservatif.
- Analisis per skenario tetap menangkap perbedaan tingkat kesulitan.
- Frekuensi drift nyata belum terdokumentasi baik, sehingga bobot sama menghindari bias tambahan.

# Catatan Pembicara: Judul

**Inti:** Penelitian ini mengevaluasi dampak operasional pilihan paradigma deployment saat terjadi configuration drift.

## Alur bicara

- Buka dengan salam dan pengenalan singkat.
- Sebutkan bahwa fokusnya adalah evaluasi empiris, bukan promosi GitOps.
- Tekankan konteks: Kubernetes, InvenioRDM, manual vs GitOps.

## Transisi

- Lanjutkan ke konteks deployment agar audiens memahami latar teknisnya.

# Catatan Pembicara: Konteks Deployment dan GitOps

**Inti:** Deployment membawa rancangan aplikasi ke state aktual kluster; GitOps muncul untuk menjaga state itu tetap deklaratif dan terlacak.

## Alur bicara

- Mulai dari definisi praktis: kode dan manifest harus menjadi layanan yang berjalan.
- Bedakan perubahan manual langsung dengan pendekatan GitOps yang memakai Git sebagai acuan.
- Tekankan bahwa GitOps bukan sekadar alat, tetapi respons terhadap kebutuhan sinkronisasi state.

## Transisi

- Setelah konteks deployment jelas, samakan istilah inti yang akan dipakai.

# Catatan Pembicara: Istilah Kunci

**Inti:** Penelitian ini bergantung pada perbedaan desired state, actual state, reconciliation, dan configuration drift.

## Alur bicara

- Jelaskan bahwa desired state adalah kondisi yang dinyatakan di manifest atau Git.
- Jelaskan bahwa actual state adalah kondisi yang benar-benar berjalan di kluster.
- Tekankan bahwa drift berarti kedua state tidak lagi sama.
- Sebutkan reconciliation sebagai proses menyelaraskan state.

## Transisi

- Setelah istilah jelas, tunjukkan dampak operasional ketika drift terjadi.

# Catatan Pembicara: Mengapa Masalah Ini Penting

**Inti:** Configuration drift membuat tim kehilangan kepastian tentang state aktual sistem.

## Alur bicara

- Mulai dari contoh perubahan cepat langsung di klaster.
- Jelaskan bahwa masalah muncul ketika dokumentasi tidak ikut berubah.
- Tekankan dampak: pemulihan lambat, akar masalah kabur, audit lemah.

## Transisi

- Masuk ke definisi formal dan bukti empiris.

# Catatan Pembicara: Definisi dan Bukti Drift

**Inti:** Drift adalah deviasi antara state aktual dan state deklaratif, dan kasusnya nyata pada Kubernetes.

## Alur bicara

- Definisikan drift secara singkat.
- Sebutkan tiga penyebab umum.
- Gunakan angka 71% sebagai bukti bahwa misconfiguration bukan kasus langka.

## Transisi

- Jelaskan bahwa GitOps hadir sebagai respons terhadap masalah ini.

# Catatan Pembicara: GitOps sebagai Respons

**Inti:** GitOps menjanjikan reconciliation otomatis antara Git dan state kluster.

## Alur bicara

- Terangkan Git sebagai desired state.
- Jelaskan agen reconciliation tanpa masuk terlalu dalam ke ArgoCD.
- Tekankan pertanyaan riset: seberapa besar dampak operasionalnya?

## Transisi

- Tunjukkan bahwa klaim praktik belum sama dengan bukti terkontrol.

# Catatan Pembicara: Klaim Industri vs Bukti

**Inti:** Klaim GitOps kuat, tetapi bukti kausal terkontrol masih terbatas.

## Alur bicara

- Akui bahwa klaim GitOps masuk akal.
- Jelaskan kelemahan umum studi sebelumnya: kualitatif, observasional, atau tanpa kontrol.
- Hindari terdengar menolak GitOps; posisikan sebagai kebutuhan pengukuran.

## Transisi

- Bawa audiens ke research gap.



# Catatan Pembicara: Research Gap

**Inti:** Penelitian ini mengisi celah metrik, kontrol variabel, analisis statistik, dan konteks akademis.

## Alur bicara

- Jangan membaca semua kotak diagram; jelaskan polanya.
- Tekankan bahwa gap ini langsung membentuk desain eksperimen.
- Sebutkan kontribusi: controlled systems experiment.

## Transisi

- Setelah gap, nyatakan rumusan masalah.

# Catatan Pembicara: Rumusan Masalah

**Inti:** Tiga dimensi utama penelitian adalah konsistensi, waktu respons, dan traceability.

## Alur bicara

- Bacakan pertanyaan utama secara perlahan.
- Hubungkan R1, R2, R3 dengan metrik yang akan dijelaskan kemudian.
- Tegaskan bahwa drift-nya terkontrol.

## Transisi

- Batasi ruang lingkup agar klaim tetap proporsional.

# Catatan Pembicara: Batasan Masalah

**Inti:** Batasan menjaga penelitian tetap terukur, dapat direplikasi, dan tidak overclaim.

## Alur bicara

- Sebutkan satu klaster, dua paradigma, tujuh skenario.
- Tekankan InvenioRDM sebagai workload uji.
- Jelaskan bahwa multi-cluster dan paradigma perantara tidak dibahas.

## Transisi

- Lanjut ke tujuan dan manfaat yang mengikuti batasan tersebut.

# Catatan Pembicara: Tujuan dan Manfaat

**Inti:** Tujuan penelitian mengikuti langsung dari R1, R2, dan R3.

## **Alur bicara**

- Jelaskan tujuan sebagai tiga keluaran pengukuran.
- Manfaat teoritis: bukti empiris.
- Manfaat metodologis dan praktis: protokol dan masukan operasional.

## **Transisi**

- Posisikan penelitian terhadap studi sebelumnya.

# Catatan Pembicara: Penelitian Terkait

**Inti:** Studi sebelumnya berguna, tetapi belum cukup untuk atribusi kausal.

## Alur bicara

- Ringkas tabel sebagai pola, bukan daftar panjang.
- Tekankan kelemahan: tanpa statistik, tanpa kontrol, atau deskriptif.
- Kaitkan langsung dengan pembeda penelitian.

## Transisi

- Masuk ke rancangan eksperimen yang mengatasi kelemahan itu.

# Catatan Pembicara: Arsitektur Eksperimen

**Inti:** Validitas internal bertumpu pada kontrol: workload sama, drift sama, klaster sama, metrik sama.

## Alur bicara

- Jelaskan dua lingkungan paralel.
- Katakan kalimat kunci: satu-satunya perbedaan adalah paradigma deployment.
- Jelaskan singkat mengapa InvenioRDM dipilih.

## Transisi

- Tunjukkan perlakuan drift yang diberikan ke dua lingkungan.

# Catatan Pembicara: Skenario Drift

**Inti:** Skenario A-G merepresentasikan kesalahan operasional umum di Kubernetes.

## Alur bicara

- Jelaskan bahwa setiap skenario diinjeksi identik.
- Tekankan randomisasi dan pengulangan bila ditanya.
- Jangan terlalu lama di setiap skenario; detail ada di backup.

## Transisi

- Lanjut ke cara respons operasional diukur.

**Inti:** Metrik didefinisikan sebelum eksperimen agar hasil dapat direplikasi dan diperiksa.

## Alur bicara

- R1: kembali atau tidak ke desired state.
- R2: waktu deteksi, pemulihan, dan identifikasi.
- R3: kelengkapan jejak perubahan.

## Transisi

- Setelah metrik, akui ancaman validitas dan mitigasinya.



**Inti:** Ancaman validitas tidak dihilangkan sepenuhnya, tetapi dikendalikan dan dibatasi klaimnya.

## Alur bicara

- Jelaskan operator tunggal dan efek pembelajaran secara terbuka.
- Tekankan runbook, timestamp otomatis, randomisasi, dan versi terkunci.
- Sebutkan bahwa hasil berlaku pada konteks yang dijelaskan.

## Transisi

- Lanjut ke analisis statistik untuk data yang dikumpulkan.

**Inti:** Mann–Whitney dipilih karena data waktu sistem belum tentu berdistribusi normal.

## Alur bicara

- Sebutkan statistik deskriptif, uji signifikansi, dan ukuran efek.
- Jelaskan  $n=10$  sebagai pengulangan sistem, bukan jumlah responden manusia.
- R3 tetap deskriptif karena sifatnya struktural.

## Transisi

- Tutup dengan kemungkinan hasil dan maknanya.

# Catatan Pembicara: Kemungkinan Hasil

**Inti:** Semua arah hasil tetap menjawab pertanyaan penelitian.

## Alur bicara

- Jika GitOps lebih baik: klaim praktik mendapat dukungan.
- Jika tidak berbeda: klaim perlu dibatasi.
- Jika manual unggul pada metrik tertentu: ada batas praktis otomatisasi.

## Transisi

- Masuk ke kalimat penutup utama.

# Catatan Pembicara: Penutup

**Inti:** Penelitian ini mengukur dampak operasional paradigma deployment, bukan membuktikan satu teknologi selalu menang.

## Alur bicara

- Ulangi kontribusi: controlled empirical evidence.
- Akhiri dengan tenang dan siap masuk tanya jawab.
- Gunakan backup slides sesuai pertanyaan penguji.

## Transisi

- Buka sesi diskusi.